

Lab 08: Query Processing and Optimizer

DBMS Course - Lab Report

A1. Create sanjose Database

```
mysql> CREATE DATABASE sanjose;
mysql> USE sanjose;
mysql> source cr_company.sql
```

After running cr_company.sql, verify the tables:

```
mysql> SHOW TABLES;
+-----+
| Tables_in_sanjose |
+-----+
| department        |
| dependent          |
| dept_locations    |
| employee           |
| project            |
| works_on           |
+-----+
6 rows in set
```

Successfully created 6 tables in the sanjose database: employee, department, dept_locations, dependent, project, and works_on.

A2. Check Catalog Database

MySQL stores metadata about all tables, columns, and indexes in the INFORMATION_SCHEMA database.

```
mysql> USE information_schema;
mysql> SHOW TABLES;
```

There are many system tables. The important ones for query optimization are TABLES, STATISTICS, and TABLE_CONSTRAINTS.

```
mysql> DESC tables;
mysql> DESC statistics;
```

Query the catalog to check EMPLOYEE table statistics:

```
mysql> SELECT table_name, table_rows, avg_row_length, data_length
-> FROM information_schema.tables
-> WHERE table_name = 'EMPLOYEE';
```

table_name	table_rows	avg_row_length	data_length
EMPLOYEE	8	81	16384

```
mysql> SELECT * FROM information_schema.statistics WHERE table_schema = 'sanjose';
```

At this point no indexes exist yet, so the statistics query returns an empty set. The catalog shows EMPLOYEE has 8 rows with average row length of 81 bytes.

A3. Collect Statistical Data using ANALYZE

This section demonstrates that catalog statistics are NOT automatically updated after data changes.

Step 1: Delete a row

```
mysql> USE sanjose;
mysql> DELETE FROM employee WHERE ssn = '333445555';
Query OK, 1 row affected
```

Step 2: Check catalog BEFORE ANALYZE

```
mysql> SELECT table_name, table_rows, avg_row_length, data_length
-> FROM information_schema.tables
-> WHERE table_name = 'EMPLOYEE';
```

table_name	table_rows	avg_row_length	data_length
EMPLOYEE	8	81	16384

Problem: table_rows still shows 8, even though we deleted one row. The catalog is stale! The optimizer would make decisions based on incorrect data.

Step 3: Run ANALYZE TABLE

```
mysql> ANALYZE TABLE employee;
+-----+-----+-----+-----+
| Table           | Op      | Msg_type | Msg_text |
+-----+-----+-----+-----+
| sanjose.employee | analyze | status   | OK        |
+-----+-----+-----+-----+
```

Step 4: Check catalog AFTER ANALYZE

```
mysql> SELECT table_name, table_rows, avg_row_length, data_length
-> FROM information_schema.tables
-> WHERE table_name = 'EMPLOYEE';
```

table_name	table_rows	avg_row_length	data_length
EMPLOYEE	7	93	16384

After ANALYZE, table_rows is updated to 7. This proves that ANALYZE TABLE forces the catalog to refresh its statistics. The query optimizer needs up-to-date statistics to choose efficient execution plans.

Restore the deleted row for subsequent exercises:

```
mysql> INSERT INTO employee VALUES ('FRANKLIN', 'T', 'WONG', '333445555', NOW(), '638 VOSS, HOUSTON TX', 'M', 40000, '888665555', 5);
```

A4. Execution Plan (EXPLAIN)

The EXPLAIN command shows how MySQL plans to execute a query without actually running it.

Before adding Primary Key

```
mysql> EXPLAIN SELECT fname, lname FROM employee;
```

id	select_type	table	type	key	rows	Extra
1	SIMPLE	employee	ALL	NULL	8	NULL

```
mysql> EXPLAIN SELECT ssn FROM employee;
```

id	select_type	table	type	key	rows	Extra
1	SIMPLE	employee	ALL	NULL	8	NULL

Both queries show type=ALL (full table scan) and key=NULL (no index used). Without any index, MySQL has no choice but to scan every row in the table.

After adding Primary Key on SSN

```
mysql> ALTER TABLE employee ADD PRIMARY KEY (ssn);
```

```
mysql> ANALYZE TABLE employee;
```

```
mysql> EXPLAIN SELECT fname, lname FROM employee;
```

id	select_type	table	type	key	rows	Extra
1	SIMPLE	employee	ALL	NULL	8	NULL

```
mysql> EXPLAIN SELECT ssn FROM employee;
```

id	select_type	table	type	key	rows	Extra
1	SIMPLE	employee	index	PRIMARY	8	Using index

Key difference: SELECT ssn now shows type=index and key=PRIMARY with 'Using index' in Extra. Since ssn IS the primary key, MySQL can satisfy the query by scanning only the index (index-only scan) without reading actual data rows. However, SELECT fname,lname still shows type=ALL because fname and lname are not in the primary key index, so MySQL must read full data rows.

A5. More Complex Query

Before adding Department Primary Key

```
mysql> EXPLAIN SELECT d.dname, SUM(e.salary)
-> FROM employee e, department d
-> WHERE e.dno = d.dnumber
-> AND e.salary >= 3000
-> GROUP BY d.dname;
```

id	select_type	table	type	key	rows	Extra
1	SIMPLE	d	ALL	NULL	3	Using temporary
1	SIMPLE	e	ALL	NULL	8	Using where; Using join buffer

Both tables use type=ALL (full table scan). No indexes are available. The GROUP BY requires a temporary table. The join uses a join buffer (block nested-loop) since there are no indexes to speed up the lookup.

After adding Department Primary Key

```
mysql> ALTER TABLE department ADD PRIMARY KEY (dnumber);
mysql> ANALYZE TABLE department;
```

Run the same query again:

```
mysql> EXPLAIN SELECT d.dname, SUM(e.salary)
-> FROM employee e, department d
-> WHERE e.dno = d.dnumber
-> AND e.salary >= 3000
-> GROUP BY d.dname;
```

id	select_type	table	type	key	rows	Extra
1	SIMPLE	e	ALL	NULL	8	Using where; Using temporary
1	SIMPLE	d	eq_ref	PRIMARY	1	NULL

Major difference: department table now shows type=eq_ref and key=PRIMARY with rows=1. This means for each employee row, MySQL directly looks up the matching department using the primary key index (one lookup per employee), instead of scanning all 3 departments. eq_ref is one of the most efficient access types.

Remove salary >= 3000 condition

```
mysql> EXPLAIN SELECT d.dname, SUM(e.salary)
-> FROM employee e, department d
-> WHERE e.dno = d.dnumber
-> GROUP BY d.dname;
```

id	select_type	table	type	key	rows	Extra
1	SIMPLE	e	ALL	NULL	8	Using temporary
1	SIMPLE	d	eq_ref	PRIMARY	1	NULL

The access methods remain the same (ALL for employee, eq_ref for department). Removing the salary condition does not change the execution strategy because there is no index on salary. The only difference is that the 'Using where' disappears from employee's Extra column since there is no longer a filter condition to

apply. All employee rows participate in the join.

A6. HINT

MySQL HINT allows forcing or ignoring specific indexes for testing execution plans.

Without HINT (normal)

```
mysql> EXPLAIN SELECT ssn FROM employee;
```

id	select_type	table	type	key	rows	Extra
1	SIMPLE	employee	index	PRIMARY	8	Using index

With IGNORE INDEX HINT

```
mysql> EXPLAIN SELECT ssn FROM employee IGNORE INDEX (PRIMARY);
```

id	select_type	table	type	key	rows	Extra
1	SIMPLE	employee	ALL	NULL	8	NULL

With IGNORE INDEX (PRIMARY), MySQL cannot use the primary key index, falling back to type=ALL (full table scan). The 'Using index' disappears from Extra. This proves the PRIMARY index was responsible for the more efficient index-only scan in the original query.

Exercise: Query Optimization using EXPLAIN and HINT

Q1: Simple Query (single table with WHERE)

Query: Find employees with salary greater than 30000

```
mysql> SELECT fname, lname, salary FROM employee WHERE salary > 30000;
```

fname	lname	salary	
FRANKLIN	WONG	40000	
JENNIFER	WALLACE	43000	
RAMESH	NARAYAN	38000	
JAMES	BORG	55000	

Execution Plan

```
mysql> EXPLAIN SELECT fname, lname, salary FROM employee WHERE salary > 30000;
```

id	select_type	table	type	key	rows	Extra
1	SIMPLE	employee	ALL	NULL	8	Using where

With HINT

```
mysql> EXPLAIN SELECT fname, lname, salary FROM employee IGNORE INDEX (PRIMARY) WHERE salary > 30000;
```

id	select_type	table	type	key	rows	Extra
1	SIMPLE	employee	ALL	NULL	8	Using where

Analysis: Both plans are identical (type=ALL, Using where). The PRIMARY key on ssn is irrelevant to this query because the WHERE condition filters on salary, not ssn. Since there is no index on salary, MySQL performs a full table scan in both cases. To improve this query, an index on salary would be needed: CREATE INDEX idx_salary ON employee(salary).

Q2: Subquery (accessing at least 2 tables)

Query: Find employees who work in the Research department

```
mysql> SELECT fname, lname FROM employee
-> WHERE dno IN (SELECT dnumber FROM department WHERE dname = 'RESEARCH');
```

Execution Plan

```
mysql> EXPLAIN SELECT fname, lname FROM employee
-> WHERE dno IN (SELECT dnumber FROM department WHERE dname = 'RESEARCH');
```

id	select_type	table	type	key	rows	Extra
1	SIMPLE	d	ref	PRIMARY	1	Using where

1	SIMPLE	e	ALL	NULL	8	Using where; Using join buffer
---	--------	---	-----	------	---	--------------------------------

With HINT

```
mysql> EXPLAIN SELECT fname, lname FROM employee
-> WHERE dno IN (SELECT dnumber FROM department IGNORE INDEX (PRIMARY)
-> WHERE dname = 'RESEARCH');
```

id	select_type	table	type	key	rows	Extra
1	SIMPLE	d	ALL	NULL	3	Using where
1	SIMPLE	e	ALL	NULL	8	Using where; Using join buffer

Analysis: Without HINT, MySQL converts the IN subquery into a semi-join and uses the PRIMARY key on department.dnumber (type=ref, rows=1). With IGNORE INDEX, department reverts to a full table scan (type=ALL, rows=3). The primary key on dnumber allows MySQL to look up the matching department directly instead of scanning all departments, reducing I/O.

Q3: Aggregate Query with GROUP BY

Query: Count employees and average salary per department

```
mysql> SELECT d.dname, COUNT(*) AS emp_count, AVG(e.salary) AS avg_salary
-> FROM employee e, department d
-> WHERE e.dno = d.dnumber
-> GROUP BY d.dname;
```

dname	emp_count	avg_salary
RESEARCH	4	33250.0000
ADMINISTRATION	3	31000.0000
HEADQUARTERS	1	55000.0000

Execution Plan

```
mysql> EXPLAIN SELECT d.dname, COUNT(*) AS emp_count, AVG(e.salary) AS avg_salary
-> FROM employee e, department d
-> WHERE e.dno = d.dnumber GROUP BY d.dname;
```

id	select_type	table	type	key	rows	Extra
1	SIMPLE	e	ALL	NULL	8	Using temporary
1	SIMPLE	d	eq_ref	PRIMARY	1	NULL

With HINT

```
mysql> EXPLAIN SELECT d.dname, COUNT(*) AS emp_count, AVG(e.salary) AS avg_salary
-> FROM employee e IGNORE INDEX (PRIMARY),
-> department d IGNORE INDEX (PRIMARY)
-> WHERE e.dno = d.dnumber GROUP BY d.dname;
```

id	select_type	table	type	key	rows	Extra
1	SIMPLE	d	ALL	NULL	3	Using temporary
1	SIMPLE	e	ALL	NULL	8	Using where; Using join buffer

Analysis: Without HINT, department uses eq_ref with PRIMARY key (rows=1 per lookup), which is very efficient. With IGNORE INDEX, both tables fall back to full table scans (type=ALL). The join changes from an index lookup to a block nested-loop join (Using join buffer), which is significantly slower for large datasets. Both plans require a temporary table for GROUP BY aggregation.

Conclusion

This lab demonstrated several key concepts in query processing and optimization:

1. System Catalog: INFORMATION_SCHEMA stores metadata (table_rows, avg_row_length, data_length) that the optimizer relies on. Running ANALYZE TABLE keeps these statistics current. Without ANALYZE, the optimizer may make suboptimal decisions based on stale data.

2. EXPLAIN Command: Reveals the optimizer's execution plan, including access types (ALL, index, eq_ref, ref), which indexes are selected, estimated row counts, and extra operations (Using where, Using temporary, Using index, Using join buffer).

3. Impact of Indexes: Adding a primary key fundamentally changes execution plans. A primary key on department.dnumber converts full table scans (ALL) into efficient direct lookups (eq_ref). An index on ssn enables index-only scans for queries that only need ssn.

4. Index Relevance: Indexes only help queries that reference the indexed column. A PRIMARY KEY on ssn does not improve queries filtering on salary. The optimizer is smart enough to ignore irrelevant indexes.

5. HINT Mechanism: IGNORE INDEX and USE INDEX allow manual control over index selection, useful for testing the impact of different execution strategies and understanding optimizer behavior.